

Documento de Requisitos — LinkTalk

Disciplina: Qualidade de Software — Versão: 1.1

João Victor Nascimento dos Santos (20231080080121)

Matias Monteiro Dantas (20221080080051)

Professor(a): Daniel Canedo

Maio de 2026

Conteúdo

1	Descrição Geral do Sistema	3
1.1	Objetivo	3
1.2	Público-alvo	3
1.3	Funcionalidades principais	3
1.4	Requisitos não funcionais (resumo)	5
1.5	Tecnologias	5
2	Diagrama de Caso de Uso	5
2.1	Atores	6
2.2	Casos de Uso Detalhados	6
3	Diagrama de Classes	9
3.1	Diagrama de classes (domínio e persistência)	9
3.2	Diagrama de classes (camadas da aplicação)	10
3.3	Principais classes e responsabilidades	11
4	DER — Diagrama Entidade-Relacionamento	12
4.1	Diagrama estrutural do banco de dados	12
4.2	Cardinalidades e regras de negócio	13
4.3	Script de criação física	14
5	Dicionário de Dados	14
5.1	Tabela <code>users</code>	15
5.2	Tabela <code>conversation</code>	15
5.3	Tabela <code>chat_group</code>	15
5.4	Tabela <code>group_members</code>	16
5.5	Tabela <code>group_admin</code>	16
5.6	Tabela <code>message</code>	16
5.7	Tabela <code>message_image</code>	18
5.8	Tabela <code>audit_log</code>	18
5.9	Sequences (Geradores de ID automáticos)	18
6	Referências ao Código e Banco de Dados	19
6.1	Estrutura do repositório	19
6.2	Instruções básicas de execução	19

1 Descrição Geral do Sistema

1.1 Objetivo

O LinkTalk foi concebido como uma aplicação web dedicada a mensagens instantâneas, projetada para facilitar a comunicação tanto de forma privada (1:1) quanto em formato de grupos. O sistema se destaca por oferecer suporte ágil e intuitivo ao envio de textos e imagens, garantindo um ambiente altamente seguro por meio de mecanismos de autenticação rigorosos e um registro detalhado de auditoria das operações. O objetivo central é fornecer uma plataforma de troca de mensagens em tempo real que seja, ao mesmo tempo, simples de utilizar e robusta em sua infraestrutura, permitindo a gestão eficiente de grupos e um controle adequado por parte dos administradores.

1.2 Público-alvo

A plataforma foi desenhada para atender a utilizadores com acesso à internet que possuam familiaridade básica com interfaces web e que necessitem de uma solução de chat síncrono sem a exigência de instalação de softwares de terceiros, rodando diretamente no navegador. Os perfis de interação com o sistema estão divididos da seguinte forma:

Perfil	Descrição
Utilizador final	Pessoas que necessitam trocar mensagens de forma ágil e constante, englobando desde estudantes até equipes de projeto e pequenos grupos de trabalho corporativo.
Administrador de grupo	Utilizador que integra um grupo de comunicação, porém detém permissões elevadas para gerir a comunidade, podendo adicionar ou remover membros, alterar o nome do grupo ou excluí-lo permanentemente do sistema.
Equipe de desenvolvimento / QA	Profissionais encarregados pela manutenção contínua, realização de testes automatizados e manuais, além da avaliação rigorosa da qualidade do sistema durante as etapas avaliativas da disciplina.

1.3 Funcionalidades principais

Para garantir a entrega de valor esperada e cobrir todos os fluxos de comunicação e gestão, o sistema foi estruturado em torno do seguinte conjunto fundamental de requisitos funcionais (RF):

ID	Funcionalidade	Descrição
RF01	Cadastro de utilizador	Registo de novas contas utilizando nome, e-mail e senha, aplicando validações rigorosas e criptografia via hash bcrypt.
RF02	Autenticação	Processo de login gerando tokens JWT (RS256) para controle seguro de sessão no cliente.
RF03	Verificação de e-mail	Consulta prévia no momento do cadastro para assegurar que o e-mail fornecido ainda não está em uso.
RF04	Conversa privada	Capacidade de iniciar e manter sessões de chat exclusivas (1:1) entre dois utilizadores.
RF05	Mensagens em tempo real	Transmissão instantânea de dados (texto e imagens) bidirecionalmente via protocolo WebSocket.
RF06	Histórico de mensagens	Funcionalidade para listar e recuperar o histórico de conversas antigas, seja em chats privados ou de grupos.
RF07	Grupos de chat	Criação de novos canais coletivos, permitindo definir o nome do grupo e adicionar seus membros fundadores.
RF08	Gestão de membros	Ferramentas exclusivas para o administrador adicionar ou remover participantes do grupo conforme necessidade.
RF09	Renomear grupo	Funcionalidade que permite ao administrador atualizar a nomenclatura do grupo.
RF10	Sair do grupo	Opção voluntária para qualquer membro se desconectar e remover sua participação ativa de um grupo.
RF11	Exclusão	Funcionalidade sensível para apagar mensagens, conversas inteiras ou deletar grupos, respeitando a hierarquia de permissões.
RF12	Busca de utilizador	Mecanismo de pesquisa de perfis no sistema utilizando o endereço de e-mail para facilitar o início de novas conversas.
RF13	Auditoria	Sistema interno de rastreamento que regista automaticamente ações relevantes do utilizador diretamente na tabela <code>audit_log</code> .
RF14	Imagens	Suporte a upload de arquivos de imagem dentro das mensagens, gerenciando o armazenamento em disco e fornecendo a URL de acesso.
RF15	Listar membros do grupo	Serviço para consultar todos os utilizadores atualmente ativos em um determinado grupo (via endpoint <code>GET /group/{id}/members</code>).

1.4 Requisitos não funcionais (resumo)

Estes requisitos definem os critérios de qualidade e as restrições arquiteturais que guiam a construção e o comportamento do LinkTalk:

ID	Requisito
RNF01	A comunicação deve ocorrer em tempo real, exigindo o uso de conexões contínuas via protocolo WebSocket.
RNF02	A segurança do tráfego e autenticação deve ser garantida com o uso de tokens JWT assimétricos (padrão RS256).
RNF03	As credenciais de acesso (senhas) devem ser armazenadas de forma segura utilizando o algoritmo de hash bcrypt, não sendo expostas em texto puro.
RNF04	A persistência e integridade dos dados da aplicação devem ser suportadas por um banco de dados relacional robusto, especificamente o PostgreSQL (versão 15).
RNF05	A API REST desenvolvida deve apresentar documentação clara e acessível diretamente no arquivo README principal do repositório do projeto.

1.5 Tecnologias

O ecossistema do LinkTalk foi construído sob uma stack tecnológica moderna, dividida estrategicamente nas seguintes camadas:

- **Frontend:** Desenvolvido com React 18 e empacotado via Vite, utilizando Redux para gerência de estado, Axios para requisições HTTP e a API nativa de WebSocket dos navegadores.
- **Backend:** Implementado em Java utilizando o framework Quarkus 3.13, aliado ao Hibernate ORM para persistência de dados, JAX-RS para a criação da API RESTful e Jakarta WebSocket para o gerenciamento das conexões em tempo real.
- **Banco de Dados:** PostgreSQL 15, provisionado de forma isolada em um ambiente de contêiner.
- **Infraestrutura:** Orquestração e padronização do ambiente local de desenvolvimento realizadas de maneira simplificada através do Docker Compose.

2 Diagrama de Caso de Uso

Nesta seção, são apresentados os limites do sistema, as interações dos utilizadores e os fluxos de valor da aplicação LinkTalk. O mapeamento baseia-se na identificação clara dos atores operacionais e na decomposição detalhada de cada caso de uso crítico.

2.1 Atores

Os atores representam as entidades externas que interagem com o LinkTalk, podendo ser papéis desempenhados por utilizadores humanos ou componentes automáticos do próprio ecossistema. A tabela abaixo detalha as responsabilidades e o escopo de atuação de cada ator:

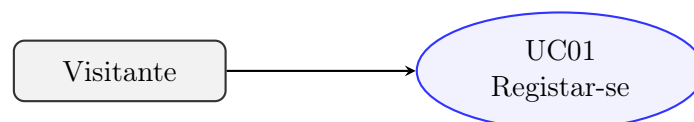
Ator	Descrição
Visitante	Utilizador que acede à plataforma web, mas que ainda não se encontra autenticado no sistema. Possui permissões estritas apenas para efetuar o registo de uma nova conta ou submeter as suas credenciais no ecrã de login.
Utilizador autenticado	Utilizador que superou com sucesso a validação de segurança e possui um token JWT ativo. Detém acesso pleno às funcionalidades de chat unilateral, busca de perfis e interações básicas de conversação.
Administrador de grupo	Um utilizador autenticado que, num contexto específico de um canal coletivo, detém privilégios elevados de gestão, sendo o responsável pela moderação de membros e integridade do grupo.
Sistema	Entidade automatizada interna encarregue de processar as regras de negócio em plano de fundo, governar a persistência, gerir as ligações síncronas e assegurar os registos de auditoria de forma transparente.

2.2 Casos de Uso Detalhados

Abaixo encontram-se descritos os oito principais casos de uso estruturais do sistema LinkTalk, acompanhados pelas respetivas especificações de comportamento e mapeamento visual de relacionamentos.

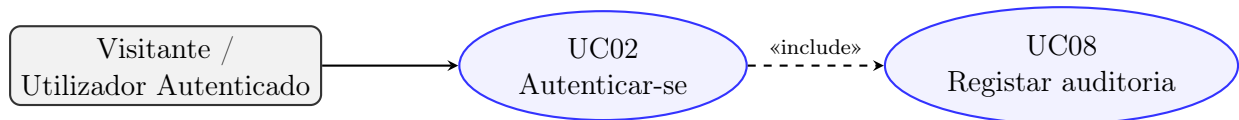
UC01 - Registar-se

- **Ator principal:** Visitante
- **Descrição:** Permite que um visitante crie uma nova conta na plataforma. O fluxo compreende a inserção de dados obrigatórios (nome completo, e-mail e palavra-passe), a validação da integridade desses dados e a subsequente encriptação da palavra-passe recorrendo ao algoritmo hash *bcrypt* antes do armazenamento seguro no banco de dados.



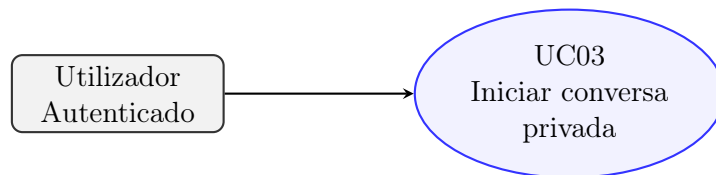
UC02 - Autenticar-se

- **Ator principal:** Visitante / Utilizador autenticado
- **Descrição:** Responsável pela validação das credenciais de acesso fornecidas. Após a confirmação positiva dos dados pelo backend, o sistema emite e retorna um token JWT estruturado sob o padrão RS256, estabelecendo o controlo de sessão seguro no lado do cliente. Este caso de uso inclui obrigatoriamente o acionamento automático do registo da ação na auditoria (UC08).



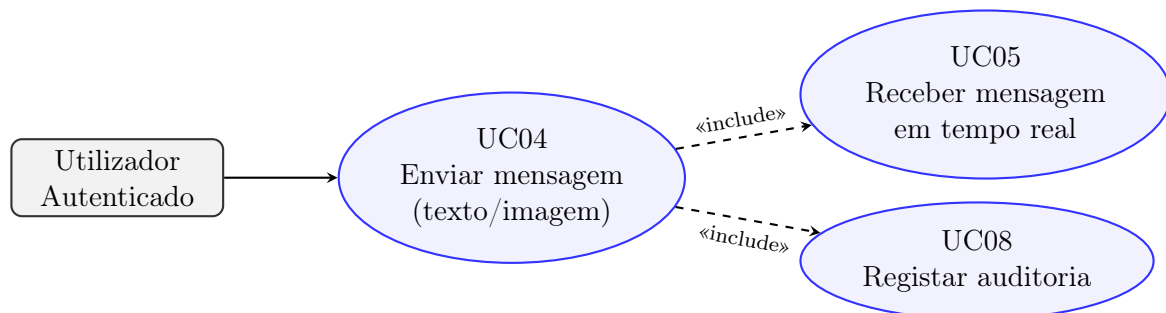
UC03 - Iniciar conversa privada

- **Ator principal:** Utilizador autenticado
- **Descrição:** Viabiliza a criação de um canal de comunicação bidirecional e exclusivo (1:1) entre dois utilizadores autenticados. O processo inicia-se com a busca ativa pelo endereço de e-mail do destinatário e culmina na abertura de uma nova instância de conversa privada no painel do utilizador.



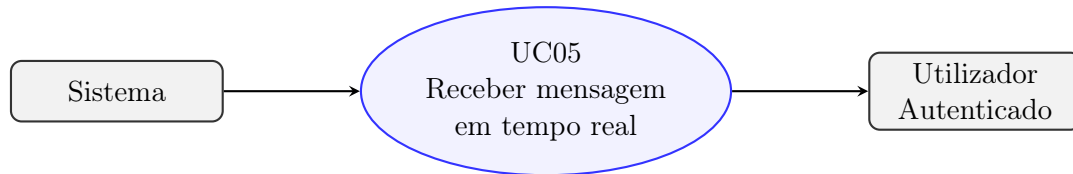
UC04 - Enviar mensagem (texto/imagem)

- **Ator principal:** Utilizador autenticado
- **Descrição:** Consiste no envio efetivo de conteúdos textuais ou ficheiros de imagem dentro de um chat ativo (seja privado ou em grupo). A transmissão de dados é processada de forma assíncrona, acionando por inclusão obrigatória a entrega da mensagem em tempo real aos destinatários (UC05) e a geração do registo de log correspondente (UC08).



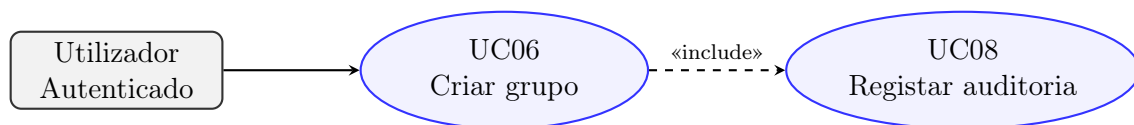
UC05 - Receber mensagem em tempo real

- **Atores:** Sistema, Utilizador autenticado
- **Descrição:** Garante que, uma vez processada a mensagem pelo backend, os pacotes de dados sejam despachados instantaneamente para as instâncias de visualização dos clientes conectados que partilham da mesma sessão ou grupo, tirando partido do modelo síncrono e persistente dos *WebSockets*.



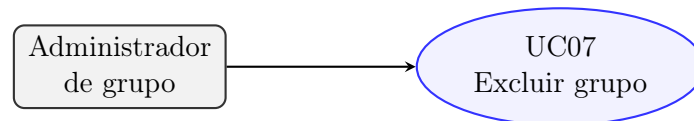
UC06 - Criar grupo

- **Ator principal:** Utilizador autenticado
- **Descrição:** Permite a um utilizador instanciar um novo canal coletivo de mensagens, definindo uma nomenclatura própria e vinculando a lista inicial de participantes. O criador do grupo é promovido automaticamente ao papel de Administrador do grupo, disparando o registo preventivo em auditoria (UC08).



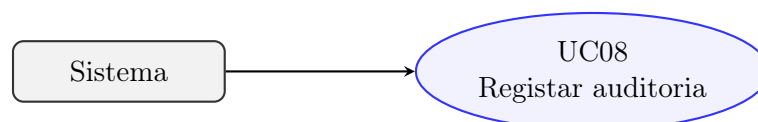
UC07 - Excluir grupo

- **Ator principal:** Administrador de grupo
- **Descrição:** Permite a remoção permanente e definitiva de um determinado grupo de chat do ecossistema LinkTalk. A operação é executada com propriedades exclusivas de eliminação em cascata (*cascade delete*), expurgando todas as mensagens, ficheiros e referências de membros associadas no banco de dados.



UC08 - Registar auditoria

- **Ator principal:** Sistema
- **Descrição:** Funcionalidade interna sob total responsabilidade da aplicação. Atua de forma independente e transparente na retaguarda do sistema, gravando com precisão cronológica e descritiva qualquer evento relevante disparado pelos utilizadores para fins de conformidade e segurança da informação.



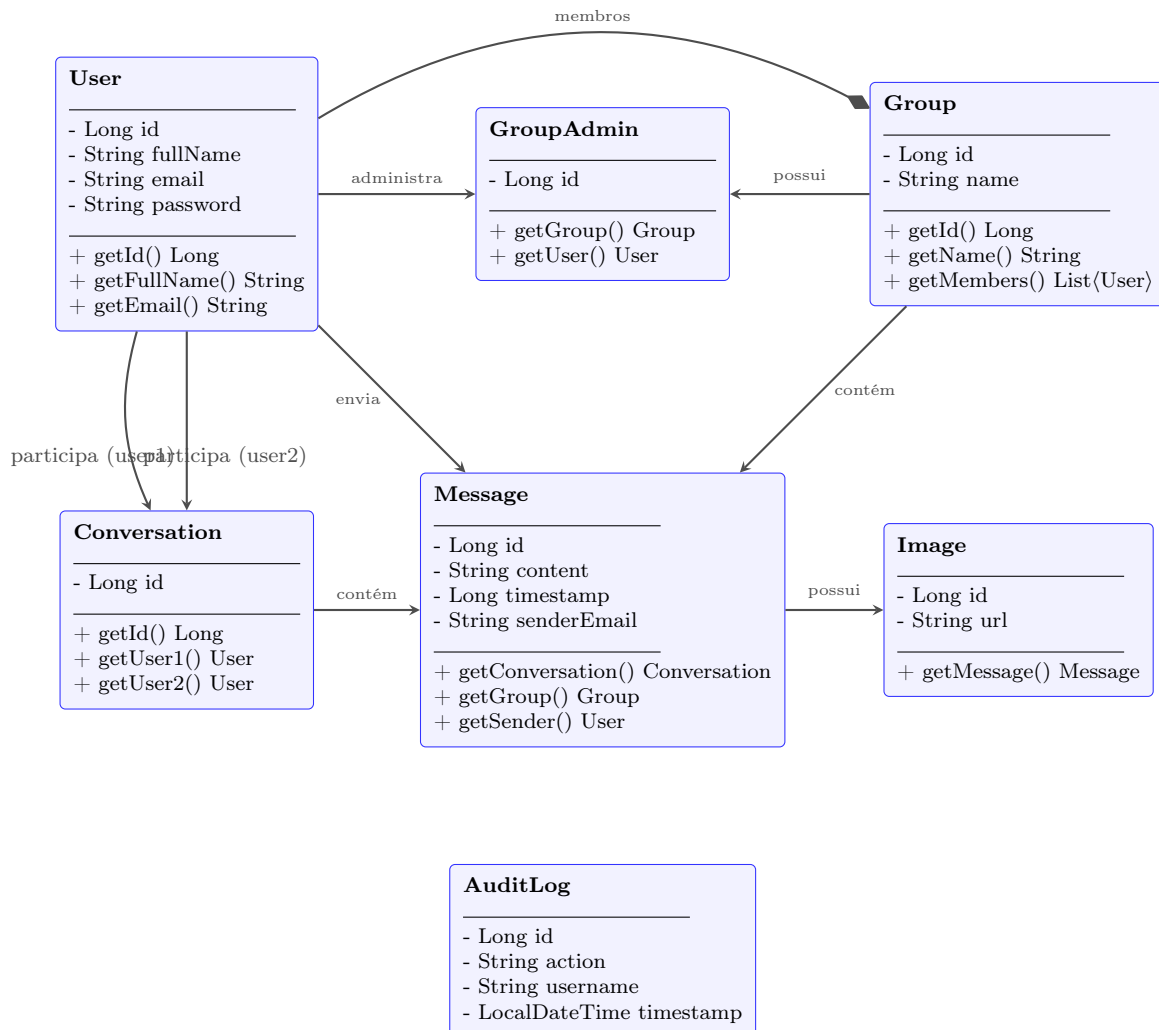
3 Diagrama de Classes

O Diagrama de Classes desempenha um papel fundamental na modelação estrutural do LinkTalk, mapeando tanto as entidades de domínio que configuram a persistência no banco de dados quanto a arquitetura lógica multicamada adotada no backend com o framework Quarkus. Esta secção está dividida entre o modelo de dados/persistência e a segregação de responsabilidades em camadas lógicas (Controladores, Camada de Negócio, Acesso a Dados e Gateways WebSocket).

3.1 Diagrama de classes (domínio e persistência)

O modelo de domínio do LinkTalk foi projetado seguindo o paradigma da programação orientada a objetos aliado à persistência relacional via Hibernate ORM. As entidades representam as tabelas físicas do PostgreSQL, priorizando associações diretas em vez de hierarquias complexas de herança, garantindo maior desempenho e simplicidade no mapeamento objeto-relacional.

Abaixo é apresentado o diagrama estrutural das classes de domínio:



Nota sobre herança: Conforme as diretrizes de design do sistema, o modelo de domínio não adota de forma alguma generalizações ou especializações entre entidades. A herança manifesta-se estritamente na árvore de exceções da aplicação (como em `InvalidLoginException`,

que estende as exceções nativas da API) e na implementação de interfaces padrão do Jakarta EE requisitadas pelo Quarkus.

Para melhor compreensão das associações e multiplicidades expostas no diagrama, a tabela seguinte detalha as regras de relacionamento:

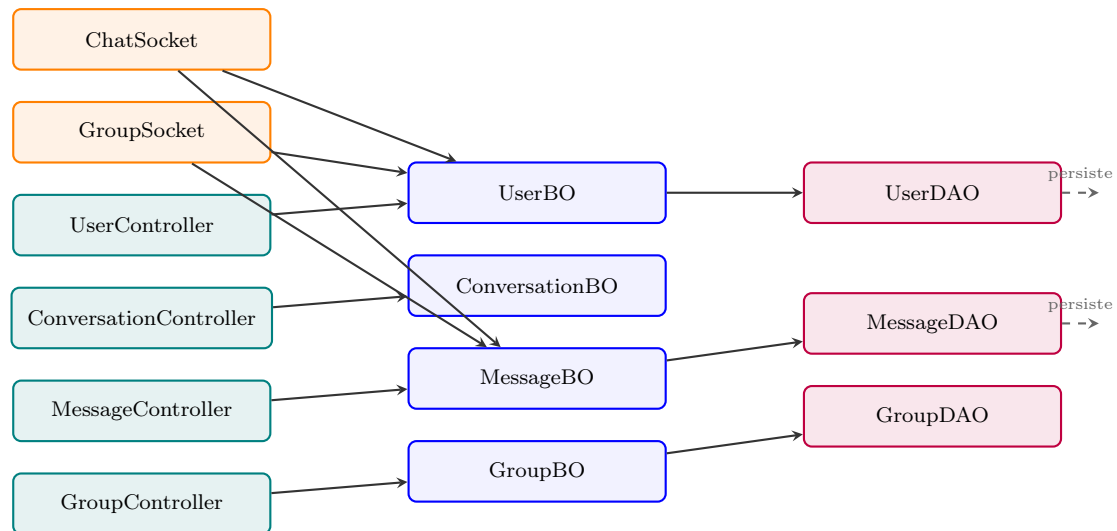
Relacionamento	Tipo UML	Cardinalidade e Regra de Negócio
User ↔ Conversation	Associação	1 utilizador pode participar ativamente de 0..* conversas privadas de forma simultânea (como remetente ou destinatário).
User ↔ Group	Associação N:M	Múltiplos membros (0..*) podem compor um grupo, assim como um utilizador pode integrar vários grupos.
User ↔ GroupAdmin	Associação	Um utilizador autenticado pode atuar como administrador de 0..* grupos específicos.
Conversation/Group → Message	Agregação	Um canal de chat (privado ou em grupo) unifica e contém historicamente de 0..* mensagens.
Message → Image	Composição	Cada mensagem enviada pode conter de forma exclusiva 0..1 imagem vinculada.

3.2 Diagrama de classes (camadas da aplicação)

A arquitetura lógica do LinkTalk baseia-se em princípios de Clean Architecture e segregação de responsabilidades. O fluxo de dados transita de forma previsível e isolada entre as seguintes camadas fundamentais da aplicação:

- **Camada de Apresentação e Redirecionamento (Controllers / Sockets):** Endpoints REST expostos via JAX-RS e canais full-duplex gerenciados via Jakarta WebSocket para conexões síncronas.
- **Camada de Negócio (Business Objects - BO):** Concentra as validações, regras de negócio e geração/verificação de tokens seguros.
- **Camada de Acesso a Dados (Data Access Objects - DAO):** Intermedeia as operações relacionais recorrendo ao ecossistema Hibernate/JPA.

O diagrama abaixo ilustra a topologia de dependências e interações entre as classes arquiteturais:



3.3 Principais classes e responsabilidades

Para consolidar o entendimento da arquitetura implementada no ecossistema do Quarkus, a tabela abaixo descreve de forma minuciosa as responsabilidades atribuídas aos principais componentes e classes do sistema:

Classe	Pacote	Responsabilidade Detalhada
User	<code>model.entity</code>	Representa a entidade de mapeamento do utilizador na plataforma, contendo dados cadastrais essenciais e chaves de validação.
Conversation	<code>model.entity</code>	Modela estruturalmente o canal de comunicação privada e bidirecional estabelecido de forma exclusiva entre dois utilizadores.
Group	<code>model.entity</code>	Define os agrupamentos ou canais coletivos de chat, gerenciando a lista de utilizadores membros e metadados como o nome da sala.
Message	<code>model.entity</code>	Abstrai o objeto de mensagem individual, contendo o corpo do texto, carimbo de data/hora (timestamp) e referências de origem/destino.
UserBO	<code>model.bo</code>	Governa as regras de negócio críticas associadas ao ciclo de vida do utilizador, validação de e-mails, criação de hash criptográfico e emissão de tokens JWT.
MessageBO	<code>model.bo</code>	Orquestra os fluxos lógicos de envio, recuperação de históricos, paginação de mensagens e controlo de exclusão física/lógica de conteúdos.
GroupBO	<code>model.bo</code>	Centraliza as regras para criação de salas coletivas, atribuição automática de privilégios de administrador e moderação de membros.

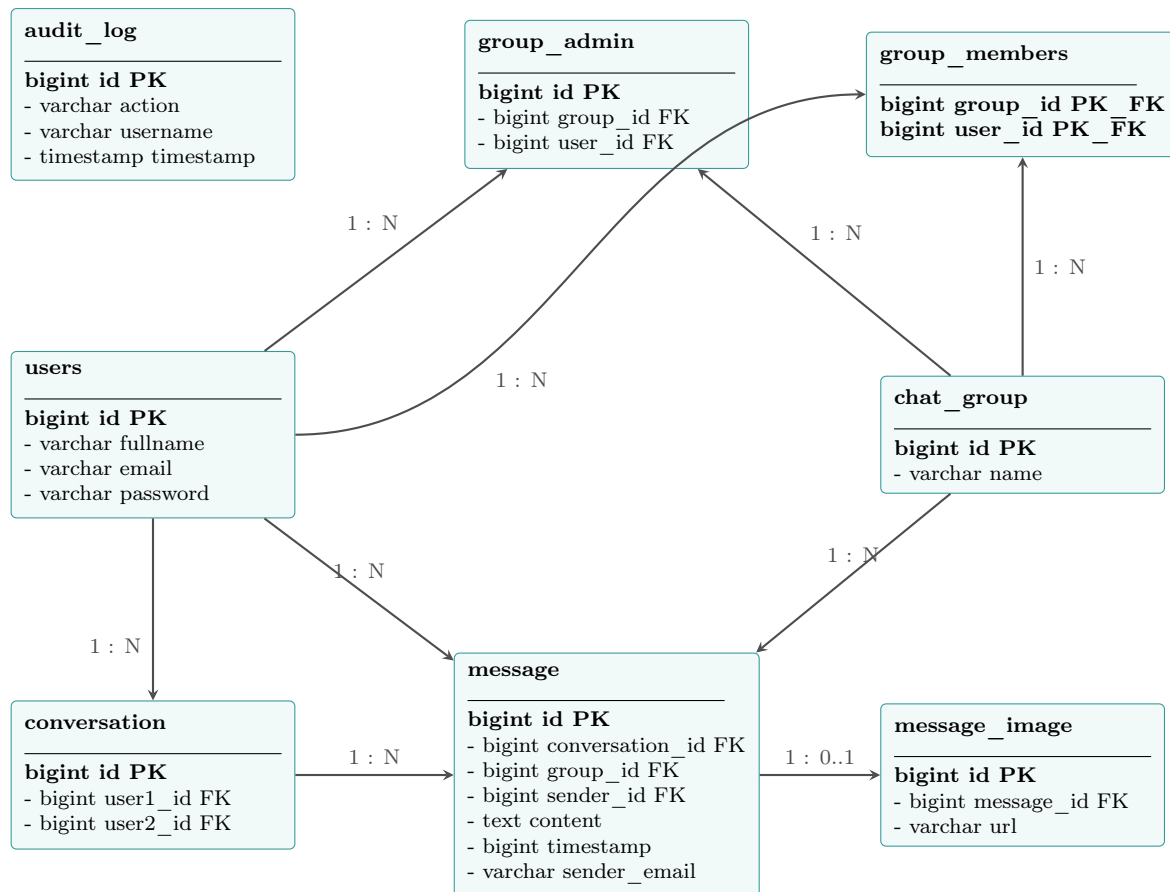
Classe	Pacote	Responsabilidade Detalhada
UserDAO	model.dao	Executa o acesso de baixo nível ao banco de dados relacional (PostgreSQL) para busca, inserção e consultas customizadas de utilizadores.
ChatSocket	socket	Provê o ciclo de vida (<i>OnOpen</i> , <i>OnMessage</i> , <i>OnClose</i>) da ligação WebSocket dedicada exclusivamente às interações de chat privado 1:1.
GroupSocket	socket	Gerencia de forma reativa os eventos síncronos de transmissão e difusão (<i>broadcast</i>) de dados para todos os participantes de um grupo conectado.

4 DER — Diagrama Entidade-Relacionamento

O Modelo Entidade-Relacionamento (ER) do LinkTalk foi concebido para garantir a integridade referencial, a consistência dos estados das conversações síncronas e a rastreabilidade total das operações administrativas. A modelação física traduz de forma direta a separação entre chats privados e canais de grupo, suportando o volume de escritas concorrentes geradas pelas ligações WebSocket.

4.1 Diagrama estrutural do banco de dados

Abaixo é apresentado o modelo relacional das tabelas geridas no PostgreSQL 15. O diagrama expõe de forma explícita as chaves primárias (PK), chaves estrangeiras (FK) e os mapeamentos associativos necessários para viabilizar as interações da aplicação:



4.2 Cardinalidades e regras de negócio

Para assegurar a consistência dos dados transacionais durante a troca de mensagens, o modelo físico impõe restrições estritas ao nível do motor de base de dados. A tabela abaixo detalha as regras de correspondência e os comportamentos esperados:

Relacionamento	Cardinalidade	Regra de Negócio Associada
users — conversation	1 : N	Uma instância de conversação privada interliga obrigatoriamente dois utilizadores distintos, atuando nos papéis de user1 e user2.
users — chat_group	N : M	Mapeado através da tabela associativa group_members. Consolida a participação multifacetada de utilizadores em múltiplos canais.
users — group_admin	N : M	Define as permissões de moderação. Aplica-se uma restrição de unicidade composta UNIQUE(group_id, user_id) para prevenir duplicidade de privilégios.
conversation — message	1 : N	Mensagens privadas contêm uma referência direta e obrigatória ao identificador da conversação de origem (conversation_id).

Relacionamento	Cardinalidade	Regra de Negócio Associada
chat_group — message	1 : N	Mensagens de grupo referenciam nativamente o identificador do grupo (<code>group_id</code>).
message	—	O modelo físico define que uma tupla deve possuir <code>conversation_id</code> OU <code>group_id</code> preenchido, nunca ambos cumulativamente.
message — message_image	1 : 0..1	Relacionamento opcional de composição. A eliminação da mensagem dispara a remoção da imagem associada por cascata.
audit_log	—	Entidade isolada e agnóstica. Regista cronologicamente as ações de auditoria sem criar amarrações com integridade referencial.

4.3 Script de criação física

A materialização do modelo de dados detalhado nesta secção é governada por um guião SQL estruturado, encarregue de criar as tabelas, índices de pesquisa e os geradores automáticos de sequências (*sequences*). O mapeamento físico completo encontra-se implementado e disponível para consulta no seguinte ficheiro do projeto:

- **Caminho do ficheiro:** `/init.sql` (localizado na raiz do repositório).

Este script é invocado de forma automática durante a fase de inicialização do contêiner Docker do PostgreSQL 15, garantindo que o ambiente de desenvolvimento local seja instanciado com a estrutura correta de forma determinística.

5 Dicionário de Dados

O Dicionário de Dados atua como o catálogo oficial e pormenorizado da estrutura física do banco de dados do LinkTalk. Esta secção documenta rigorosamente as tabelas, os tipos de dados adotados (suportados nativamente pelo PostgreSQL 15), as restrições de nulidade e a identificação precisa das chaves primárias (PK) e estrangeiras (FK), além das regras de negócio atreladas a cada campo.

Convenções adotadas nas tabelas a seguir:

- **PK:** Chave Primária (*Primary Key*)
- **FK:** Chave Estrangeira (*Foreign Key*)
- **UK:** Chave Única (*Unique Key*)

5.1 Tabela users

Esta entidade é o núcleo do sistema de identidade da aplicação. Armazena as informações essenciais de perfil e as credenciais de acesso (criptografadas) de todos os utilizadores registados na plataforma.

Campo	Tipo	Nulo	Chave	Descrição
id	BIGINT	Não	PK	Identificador único do utilizador, gerado automaticamente pela sequence <code>users_id_seq</code> .
fullname	VARCHAR(255)	Não	—	Nome completo do utilizador, destinado à exibição nas interfaces de chat.
email	VARCHAR(255)	Não	UK*	Endereço de e-mail utilizado para o login; deve ser estritamente único em toda a aplicação.
password	VARCHAR(255)	Não	—	Hash criptográfico da palavra-passe gerado pelo algoritmo <i>bcrypt</i> ; a senha em texto puro nunca é persistida.

* **Nota de Implementação:** O script atual (`init.sql`) não declara explicitamente a restrição `UNIQUE` no campo `email`; a unicidade é atualmente garantida e blindada pela regra de negócio na camada lógica (`UserBO.register()`). Recomenda-se a adição da *constraint* `UNIQUE (email)` a nível de banco de dados nas futuras iterações do projeto.

5.2 Tabela conversation

Entidade responsável por instanciar e gerir os canais de comunicação estritamente privados (1:1), atuando como a ponte de ligação exclusiva entre dois utilizadores distintos.

Campo	Tipo	Nulo	Chave	Descrição
id	BIGINT	Não	PK	Identificador único da sessão de conversa privada.
user1_id	BIGINT	Não	FK → <code>users.id</code>	Referência ao primeiro participante da conversa.
user2_id	BIGINT	Não	FK → <code>users.id</code>	Referência ao segundo participante da conversa.

5.3 Tabela chat_group

Guarda os metadados primários dos canais de comunicação coletivos, permitindo que múltiplos utilizadores interajam num mesmo ambiente partilhado.

Campo	Tipo	Nulo	Chave	Descrição
id	BIGINT	Não	PK	Identificador único do grupo de chat.
name	VARCHAR(255)	Não	—	Nomenclatura ou título de exibição do grupo, definido e editável pelo administrador.

5.4 Tabela group_members

Tabela puramente associativa encarregue de materializar o relacionamento N:M (muitos-para-muitos) entre as entidades `users` e `chat_group`.

Campo	Tipo	Nulo	Chave	Descrição
group_id	BIGINT	Não	PK, FK → <code>chat_group.id</code>	Identificador do grupo ao qual o utilizador se encontra vinculado.
user_id	BIGINT	Não	PK, FK → <code>users.id</code>	Identificador do utilizador que atua como membro do referido grupo.

5.5 Tabela group_admin

Esta tabela segrega a responsabilidade de gestão de privilégios dentro dos grupos. Ela eleva um membro comum à categoria de administrador, concedendo-lhe poderes de moderação.

Campo	Tipo	Nulo	Chave	Descrição
id	BIGINT	Não	PK	Identificador único do registo de delegação de administração.
group_id	BIGINT	Não	FK → <code>chat_group.id</code>	Identificador do grupo onde o privilégio será exercido.
user_id	BIGINT	Não	FK → <code>users.id</code>	Identificador do utilizador promovido ao papel de administrador.

Restrição Estrutural: Aplica-se a regra de unicidade `UNIQUE (group_id, user_id)` para assegurar que um mesmo utilizador não possa possuir múltiplos registos redundantes de administração no mesmo grupo.

5.6 Tabela message

A entidade de maior volume transacional do sistema. Armazena o conteúdo, a autoria e a cronologia de todas as mensagens traficadas, independentemente de o destino ser um chat privado ou um grupo.

Campo	Tipo	Nulo	Chave	Descrição
id	BIGINT	Não	PK	Identificador sequencial da mensagem.
conversation_id	BIGINT	Sim	FK → conversation.id	Chave da conversa privada. Preenchido estritamente quando o envio ocorre num contexto 1:1.
group_id	BIGINT	Sim	FK → chat_group.id	Chave do grupo. Preenchido estritamente quando o envio ocorre num chat coletivo.
sender_id	BIGINT	Não	FK → users.id	Identificador do autor ou remetente da mensagem.
content	TEXT	Não	—	Corpo textual da mensagem. Pode apresentar-se em formato de <i>string</i> vazia caso a mensagem contenha apenas uma imagem.
timestamp	BIGINT	Não	—	Carimbo de data e hora do envio armazenado em formato numérico

5.7 Tabela `message_image`

Gerencia de forma desacoplada os recursos multimídia atrelados às mensagens, guardando os caminhos lógicos para o armazenamento físico das imagens no servidor.

Campo	Tipo	Nulo	Chave	Descrição
id	BIGINT	Não	PK	Identificador único do recurso de imagem.
url	VARCHAR(255)	Não	—	Rota relativa ou URL absoluta para aceder ao ficheiro estático hospedado no servidor (ex.: <code>/images/{nome}</code>).
message_id	BIGINT	Sim	FK → <code>message.id</code>	Referência cruzada para a mensagem de texto à qual este anexo multimídia pertence.

5.8 Tabela `audit_log`

A infraestrutura de auditoria em base de dados. Regista de forma perene os eventos sensíveis da plataforma, atuando como um passaporte de segurança para rastrear anomalias e ações dos utilizadores.

Campo	Tipo	Nulo	Chave	Descrição
id	BIGINT	Não	PK	Identificador sequencial do registo no <i>log</i> de auditoria.
action	VARCHAR(255)	Não	—	Classificação canónica ou nome da ação disparada (por exemplo, <code>LOGIN_REQUEST</code> , <code>GROUP_CREATED</code>).
username	VARCHAR(255)	Não	—	Assinatura do responsável pelo evento, geralmente representada pelo e-mail ou identificador de sessão.
timestamp	TIMESTAMP	Não	—	Registo exato do instante cronológico (data e hora) em que a operação ocorreu no sistema.

5.9 Sequences (Geradores de ID automáticos)

O banco de dados recorre ao mecanismo de sequências nativo do PostgreSQL para garantir a geração atómica, segura e performática dos identificadores numéricos das chaves primárias.

Sequence	Tabela Destino / Associada
users_id_seq	users
conversation_id_seq	conversation
group_chat_id_seq	chat_group
group_admin_id_seq	group_admin
message_id_seq	message
message_image_id_seq	message_image
audit_log_id_seq	audit_log

6 Referências ao Código e Banco de Dados

Para assegurar a reprodutibilidade do ambiente de desenvolvimento e facilitar a manutenção contínua do sistema, esta secção mapeia a topologia do repositório de código-fonte e documenta os procedimentos fundamentais para a orquestração e execução local da aplicação LinkTalk.

6.1 Estrutura do repositório

O código-fonte do LinkTalk encontra-se organizado de forma modular, segregando claramente as responsabilidades entre a infraestrutura base, a interface de utilizador (frontend) e a camada de serviços (backend). A árvore de diretórios principal apresenta a seguinte disposição:

```
linktalk-main/
|-- init.sql           # Script de criação do banco (PostgreSQL)
|-- docker-compose.yml # Container do PostgreSQL
|-- README.md         # Instruções de execução
|-- docs/
|   '-- Documento-de-Requisitos.md # Este documento
|-- backend/          # API Quarkus (Java 17)
'-- frontend/         # Interface React (Vite)
```

6.2 Instruções básicas de execução

Para instanciar o ecossistema do LinkTalk num ambiente de desenvolvimento local, é imperativo seguir a sequência de inicialização descrita abaixo. Certifique-se de que o host de execução possui as ferramentas essenciais previamente configuradas (tais como Docker, Node.js e JDK 17).

1. **Inicializar o Banco de Dados:** Aceda ao diretório raiz do projeto e orquestre a subida do contentor do PostgreSQL executando o seguinte comando:

```
docker compose up -d
```

2. **Arrancar o Backend:** Navegue até à diretoria `backend` e inicie a API Quarkus em modo de desenvolvimento contínuo (*dev mode*). Utilize o script correspondente ao seu sistema operativo:

- **Ambientes Windows:** `mvnw.cmd quarkus:dev`
- **Ambientes Linux/macOS:** `./mvnw quarkus:dev`

3. **Arrancar o Frontend:** Numa nova janela ou aba de terminal, navegue até à diretoria `frontend`, instale as dependências atreladas ao projeto e inicie o servidor de desenvolvimento do Vite:

```
npm install → npm run dev
```

4. **Acessar a Aplicação:** Com a base de dados provisionada e os serviços em pleno funcionamento, o sistema estará recetivo para interações através dos seguintes pontos de entrada:

- **Interface Web (Frontend):** Aceda a `http://localhost:5173` através do seu navegador.
- **Serviços REST e WebSockets (API):** Disponíveis operacionalmente no endereço `http://localhost:8081`.

Nota Final: Para consultar informações suplementares, detalhes exaustivos sobre as portas de rede mapeadas e as credenciais padrão exigidas para autenticação na base de dados, por favor, recorra ao ficheiro `README.md` situado na raiz do repositório do projeto.